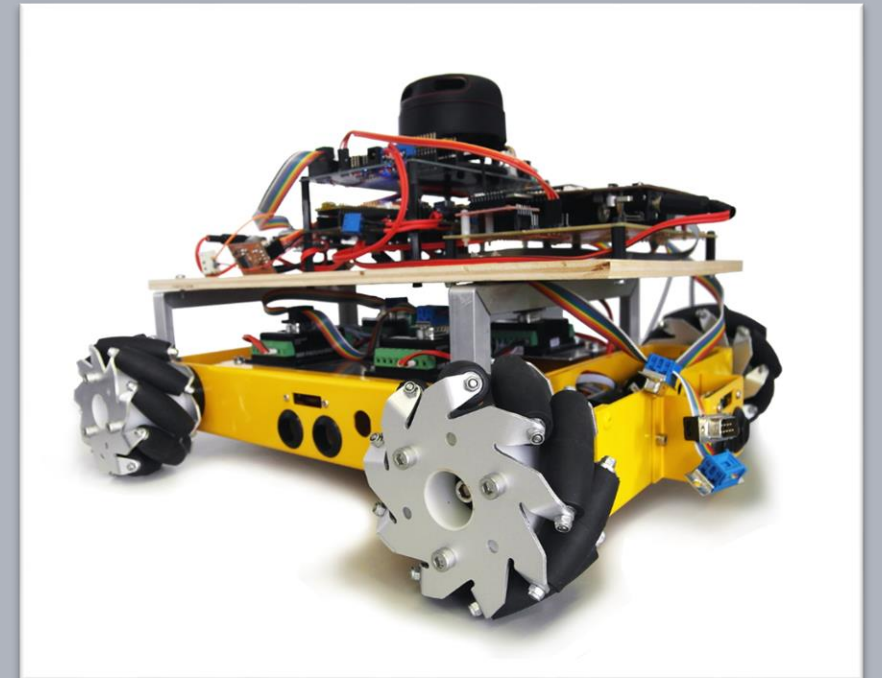


Advanced Embedded Architectures and Applications

Ultrasonic Sensor Integration and Filtering

Prof. Dr.-Ing. Peter Fromm



Content

- The Ultrasonic Sensor
- Driver development
- Integration into the RTE
- Digital Filtering
- An extended Driving Algorithm
- The mystery of Stack Allocation



Working Principle



Picture: <https://components101.com/ultrasonic-sensor-working-pinout-datasheet>

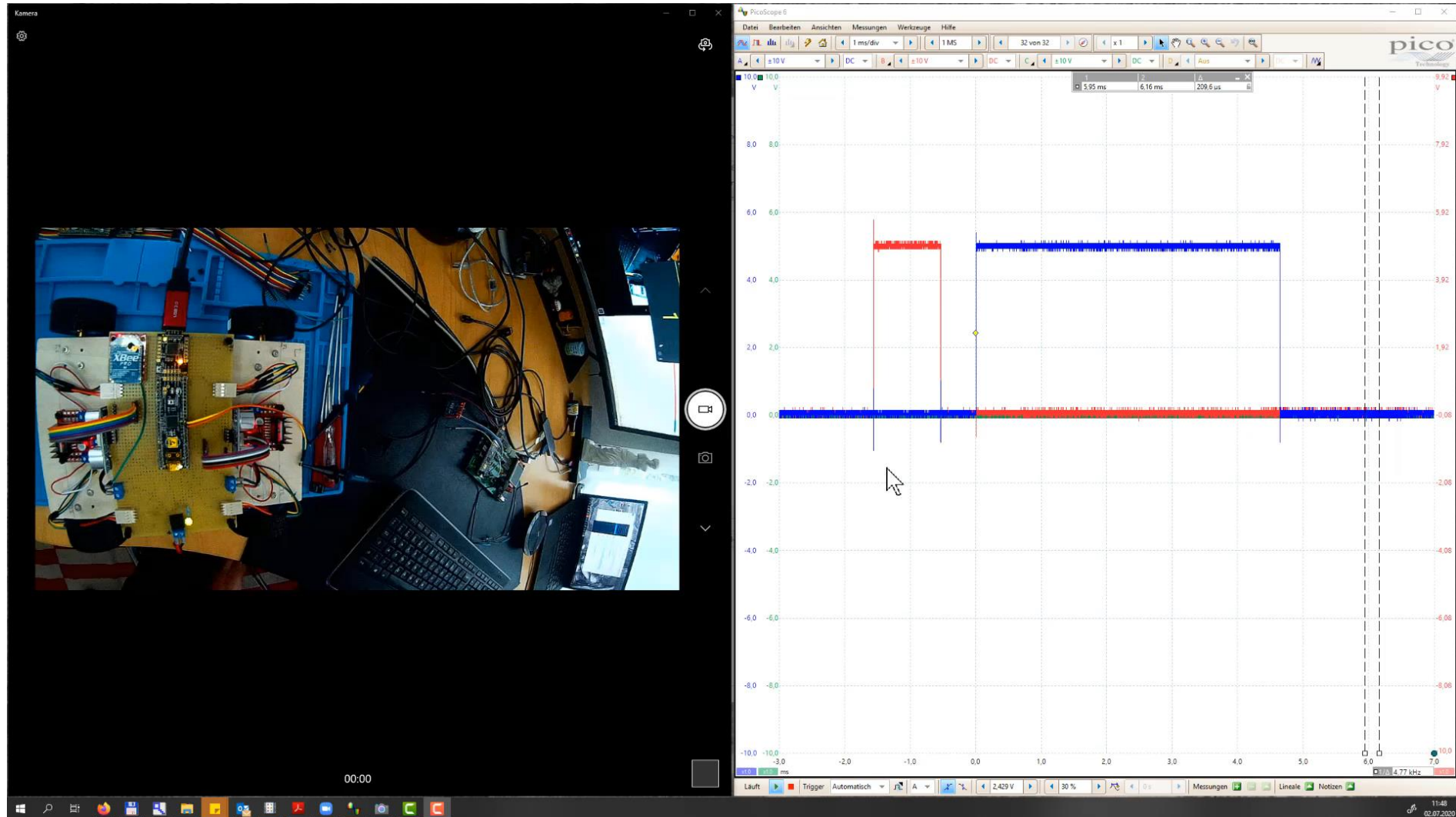
$$\Delta t = \frac{2 \times \text{distance}}{c_s} \quad (1)$$

$$\text{distance} = \frac{\Delta t \times c_s}{2} \quad (2)$$

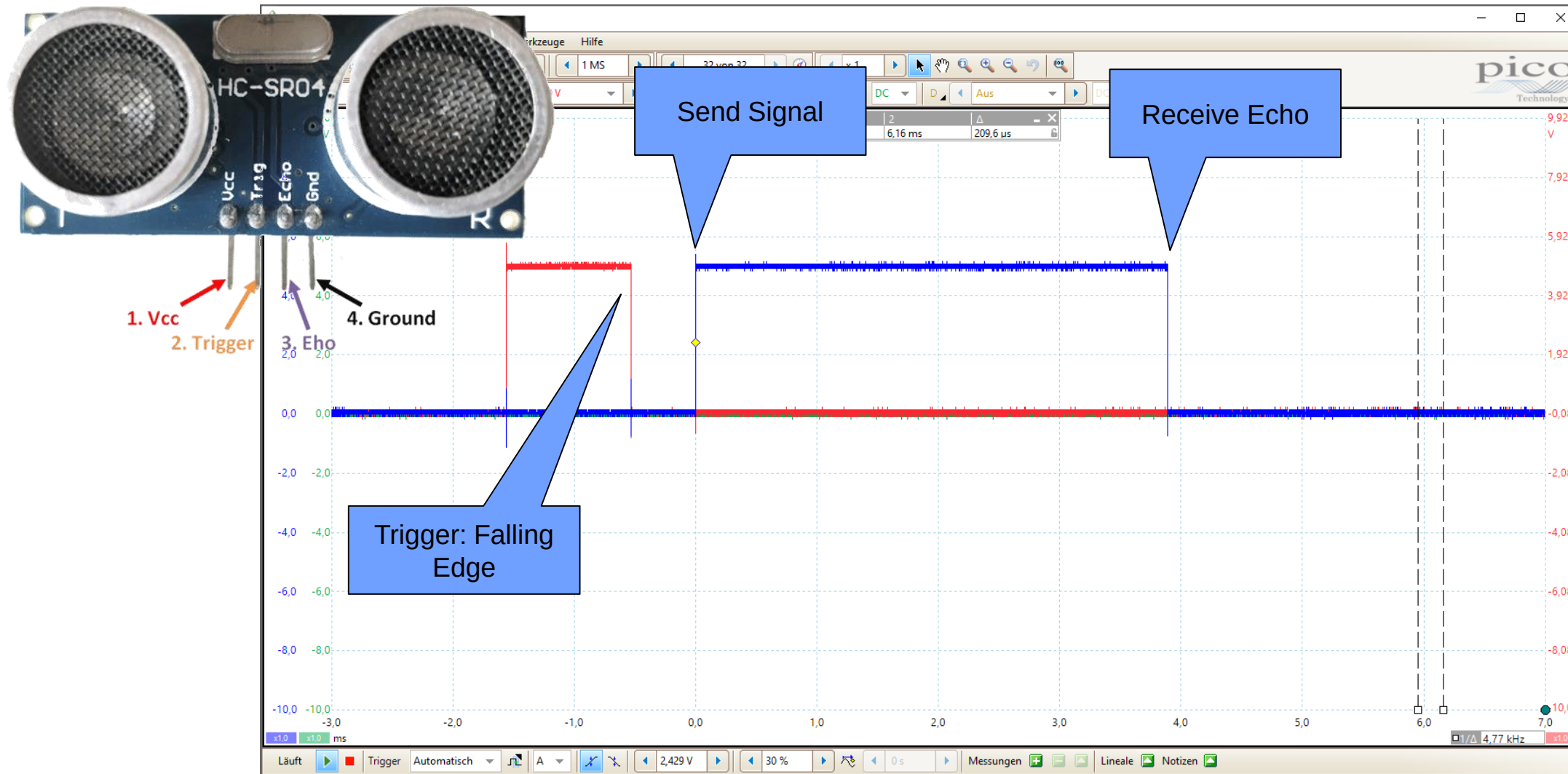
$$c_s = (331.5 \times 0.6 \frac{\vartheta}{^\circ\text{C}}) \frac{\text{m}}{\text{s}} \quad (3)$$

$$\text{distance} \approx \Delta t \times 17,15 \frac{\text{cm}}{\text{ms}}$$

Working Principle



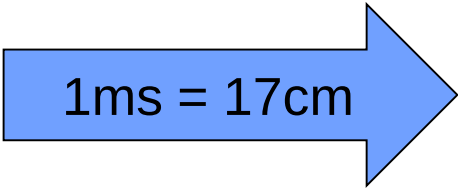
Driver Design



Some preparational thoughts

- Which precision do we require for the time measurement?
- What are the shortest and longest time periods we have to measure?
- Implementation as synchronous or asynchronous driver?
- Can we fire more than one sensor at the same time?
- What would be the fastest cycle time for sensor values?

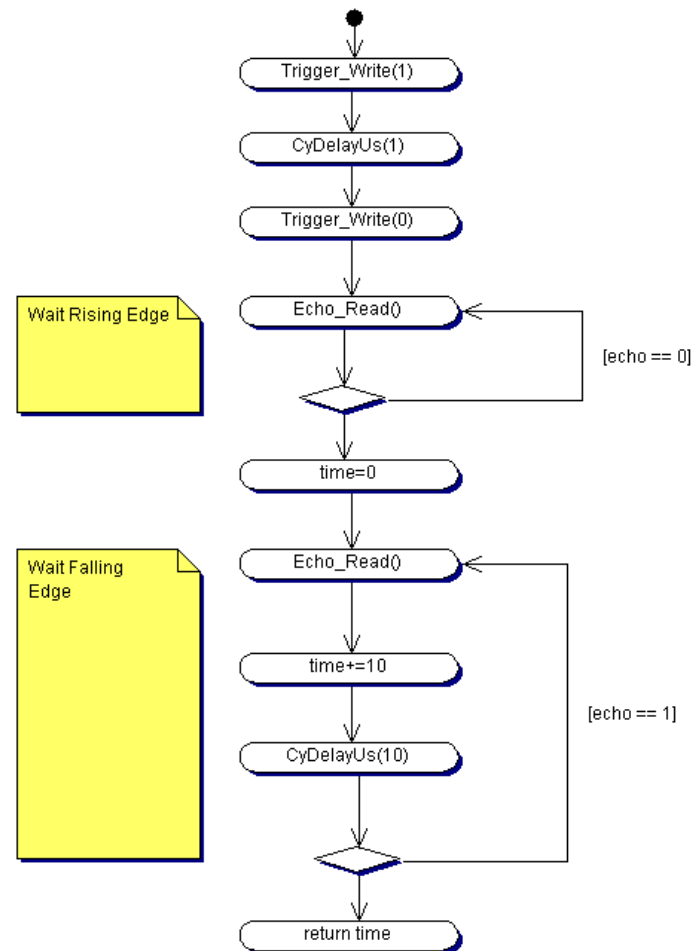
$$distance \approx \Delta t \times 17,15 \frac{cm}{ms}(4)$$


$$1ms = 17cm$$

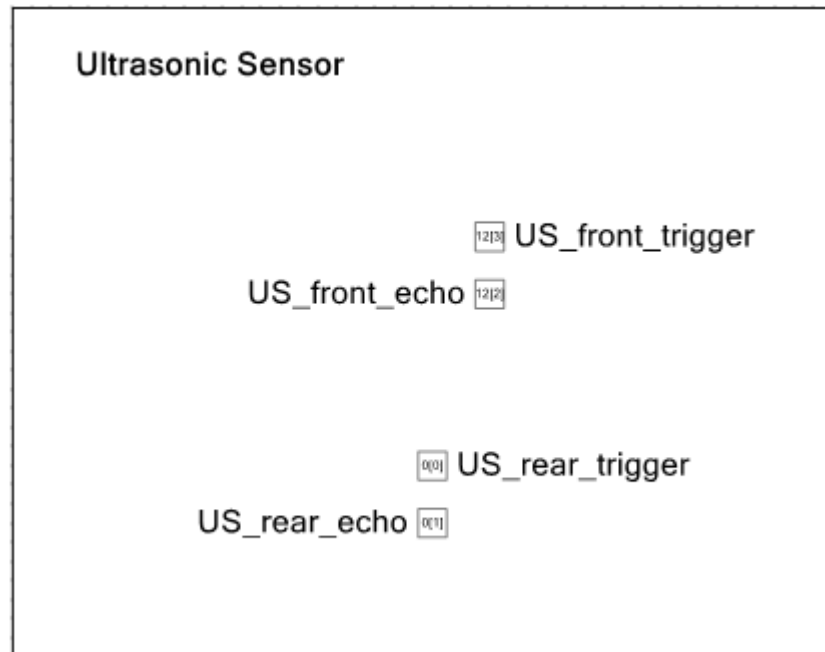
Entfernung zum Objekt [cm]	Laufzeit [ms] bei 20°C	Laufzeit [ms] bei 0°C
2	0,117	0,121
10	0,583	0,603
50	2,915	3,017
100	5,831	6,033
200	11,662	12,066
300 (max. Dist.)	17,492	18,100



First Design Idea: Synchronous Driver using Software Polling



Synchronous Polling Driver Hardware and Software



```
/**
 * A brute force polling implementation of the driver
 * \param US_id_t id : [IN] The sensor to be fired
 * \param uint16_t* const distance : [OUT] Distance in cm
 * @return RC_SUCCESS
 */
RC_t US_GetDistance_poll(US_id_t id, uint16_t* const distance)
{
    //Get the MCAL function
    US_trigger trigger = US_ports[id].m_trigger_func;
    US_echo echo = US_ports[id].m_echo_func;

    if (0 != trigger && 0 != echo)
    {
        //Fire the trigger
        trigger(1);
        CyDelayUs(10);
        trigger(0);

        //Wait until rising edge of echo signal
        while(1)
        {
            if (1 == echo()) break;
        }

        //wait until falling edge and count time - with an unknown precision
        uint32_t time = 0;
        while(1)
        {
            if (0 == echo()) break;

            CyDelayUs(10);
            time += 10;
        }

        *distance = US_calcDistance(time);
        LOG_I("US_GetDistance_poll", "Time = %d Distance = %d", time, *distance);
    }
    else
    {
        *distance = US_INVALIDDISTANCE;
        LOG_E("US_GetDistance_poll", "no pin config");
    }
}
```


Synchronous Polling Driver Validation

Pro

- Very simple hardware configuration
- Straight forward software
- Trigger and Echo signal are handled in a single function – making mapping very easy

Con

- Too long blocking time (up to 18ms)
- Will spoil realtime behavior of cyclic runnables having a higher frequency
- Will spoil software clock of the RTE
- Placement in a lower priority task is no option – why?

Improvement 1: Implementing a asynchronous driver using a hardware timer

General idea

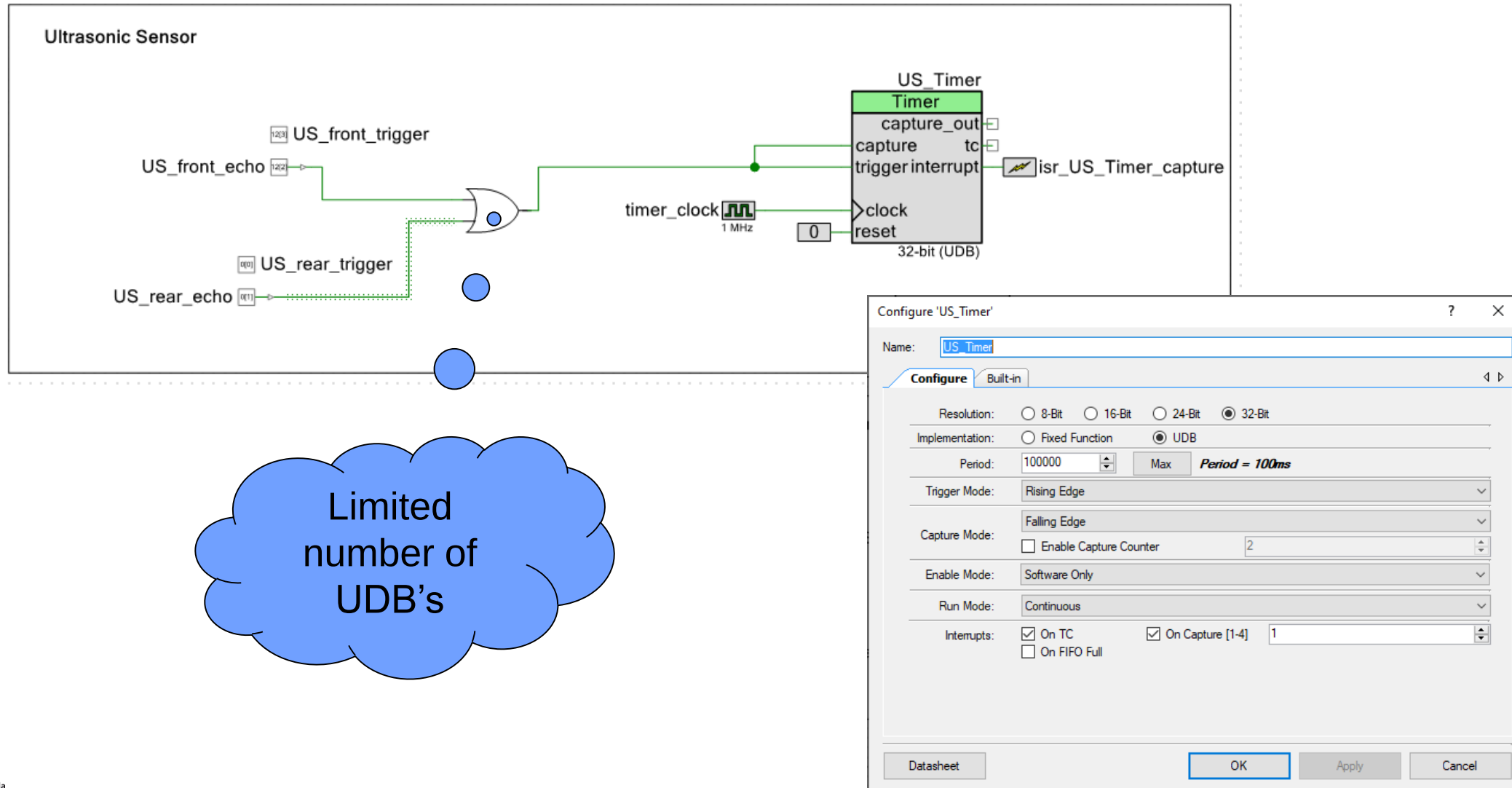
- The pullport command will only fire the trigger pulse
- A hardware counter will measure the time between rising and falling edge
- An ISR will write the capture value into the signal

Good article: <https://iotexpert.com/2017/05/17/psoc-5-timer-circuit-debug/>

Very short blocking time

Code is spread over two functions, which need to be synchronized.

Improvement 1: Implementing a asynchronous driver using a hardware timer



Pullport

MCAL drivers are attributes of the object.

```
/**
 * Fire function of the Ultrasonic Sensor
 * \param US_id_t : [IN] The sensor to be fired
 * @return RC_SUCCESS
 */
RC_t US_Fire(US_id_t id)
{
    //Get the MCAL function
    US_trigger trigger = US_ports[id].m_trigger_func;
    US_echo echo = US_ports[id].m_echo_func;

    if (0 != trigger)
    {
        trigger(1);
        CyDelayUs(10);
        trigger(0);
    }
    else
    {
        LOG_E("US_Fire", "no pin config - id = %d", id);
    }
}
```

```
const US_port_cfg_t US_ports[US_MAX] =
{
    /* US_NONE */ { 0, 0},
    /* US_FRONT */ {US_front_trigger_Write, US_front_echo_Read},
    /* US_REAR */ {US_rear_trigger_Write, US_rear_echo_Read},
    /* US_RIGHT */ { 0, 0},
    /* US_LEFT */ { 0, 0}
};
```

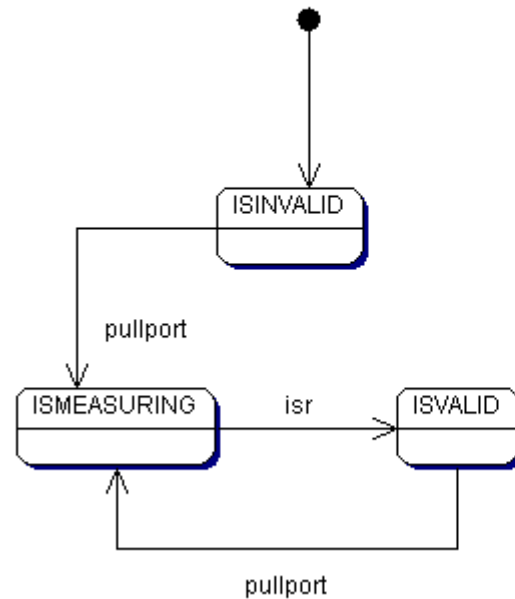
Challenge

- The interrupt will be fired
- Easy going for a single sensor. But we have more than one
- How do we know to which signal object the time belongs?
- We could use an individual timer and ISR for every sensor, but this would be rather expensive...



Solution

- Not 100% reliable, but somehow elegant



As long as the signals are updated one after the other, only one will be in the ISMEASURING state at the same time

Resulting Code

```
/**
 * Default IN driver API - may be deleted if not required
 */
inline RC_t SIGNAL_ULTRASONICSENSOR_driverIn(SIGNAL_ULTRASONICSENSOR_data_t *const data)
{
    //LOG_I("SIGNAL_ULTRASONICSENSOR_driverIn","id %d",data->m_id);

    data->m_status = US_ISMEASURING;

    //Testcase 1 - Just firing
    #if TESTCASE == 1
    US_Fire(data->m_id);
    data->m_status = US_ISINVALID;
    #endif

    //Testcase 2 - Polling
    #if TESTCASE == 2
    uint16_t distance = 0;
    US_GetDistance_poll(data->m_id,&distance);
    data->m_distance = distance;
    data->m_status = US_ISVALID;
    #endif

    //Testcase 3 - ISR
    #if TESTCASE == 3
    US_Fire(data->m_id);
    #endif

    return RC_SUCCESS;
}
```

```
ISR2(isr_US_Timer_capture)
{
    uint16_t distance;
    uint8_t statusFlag = US_Timer_ReadStatusRegister();

    if(statusFlag & US_Timer_STATUS_TC)
    {
        LOG_E("isr_US_Timer_capture","Error TC Trigger");
    }
    else if (statusFlag & US_Timer_STATUS_CAPTURE)
    {
        US_ReadDistance_isr(&distance);

        //Now we have to write the distance to the correct signal
        //In the pullport function, the corresponding signal status has been set to US_ISMEASURING
        //The User has to make sure, that only one signal is being fired at the same time
        //Additional locks may be added to the pullport, i.e. return an error if one of the other signals is in the IS_MEASURING state.
        //But this seems a bit hacky...
        //Todo: Find a better solution

        //LOG_I("Isr","D = %d",distance);

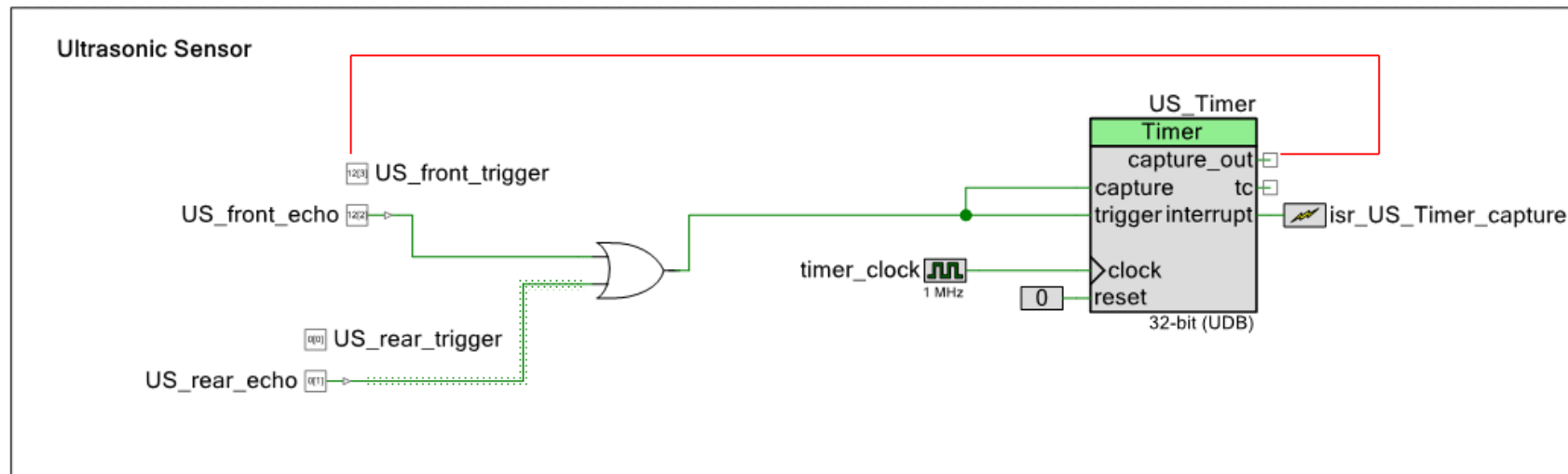
        SIGNAL_ULTRASONICSENSOR_data_t us = SIGNAL_ULTRASONICSENSOR_INIT_DATA;

        //Check for front sensor
        us = RTE_SIGNAL_ULTRASONICSENSOR_get(&SO_USFRONT_signal);
        if (us.m_status == US_ISMEASURING)
        {
            us.m_distance = distance;
            us.m_status = US_ISVALID;
            RTE_SIGNAL_ULTRASONICSENSOR_set(&SO_USFRONT_signal, us);
        }

        //Check for rear sensor
        us = RTE_SIGNAL_ULTRASONICSENSOR_get(&SO_USREAR_signal);
        if (us.m_status == US_ISMEASURING)
        {
            us.m_distance = distance;
            us.m_status = US_ISVALID;
            RTE_SIGNAL_ULTRASONICSENSOR_set(&SO_USREAR_signal, us);
        }
    }
}
```

Further Ideas

- We could make the timer fire the trigger automatically
- We would get the data as fast as possible, i.e. a higher sample rate at lower distances
- But the echoes from the front and rear sensor might interfere
- And we would need extra timers per channel



Filters

Remember the somewhat jumpy signal of the echo during far distances?

- Moving Average Filter
- Median Filter
- Min/Max Filter
- Plausibility validation / invalidation of signals
- Kalman Filters
- ...



Moving Average

- Simple filter to decrease the impact of white noise
- Acts as a low pass filter, i.e. high frequency parts of the signal are suppressed
- Implementation as FIFO buffer, e.g. ringbuffer
- Extension: Weighted moving average

$$CMA_n = \frac{x_1 + \dots + x_n}{n}.$$

$$\begin{aligned} CMA_{n+1} &= \frac{x_{n+1} + n \cdot CMA_n}{n+1} \\ &= \frac{x_{n+1} + (n+1-1) \cdot CMA_n}{n+1} \\ &= \frac{(n+1) \cdot CMA_n + x_{n+1} - CMA_n}{n+1} \\ &= CMA_n + \frac{x_{n+1} - CMA_n}{n+1} \end{aligned}$$



Median Filter

The median filter is a non-linear digital filtering technique, often used to remove noise from an image or signal.

The main idea of the median filter is to run through the signal entry by entry, replacing each entry with the median of neighboring entries. The pattern of neighbors is called the "window", which slides, entry by entry, over the entire signal. For one-dimensional signals, the most obvious window is just the first few preceding and following entries, whereas for two-dimensional (or higher-dimensional) data the window must include all entries within a given radius or ellipsoidal region (i.e. the median filter is not a separable filter).

Good to filter out invalid spikes of a signal.

$$x = (2, 3, 80, 6).$$

So, the median filtered output signal y will be:

$$y_1 = \text{med}(2, 3, 80) = 3,$$

$$y_2 = \text{med}(3, 80, 6) = \text{med}(3, 6, 80) = 6,$$

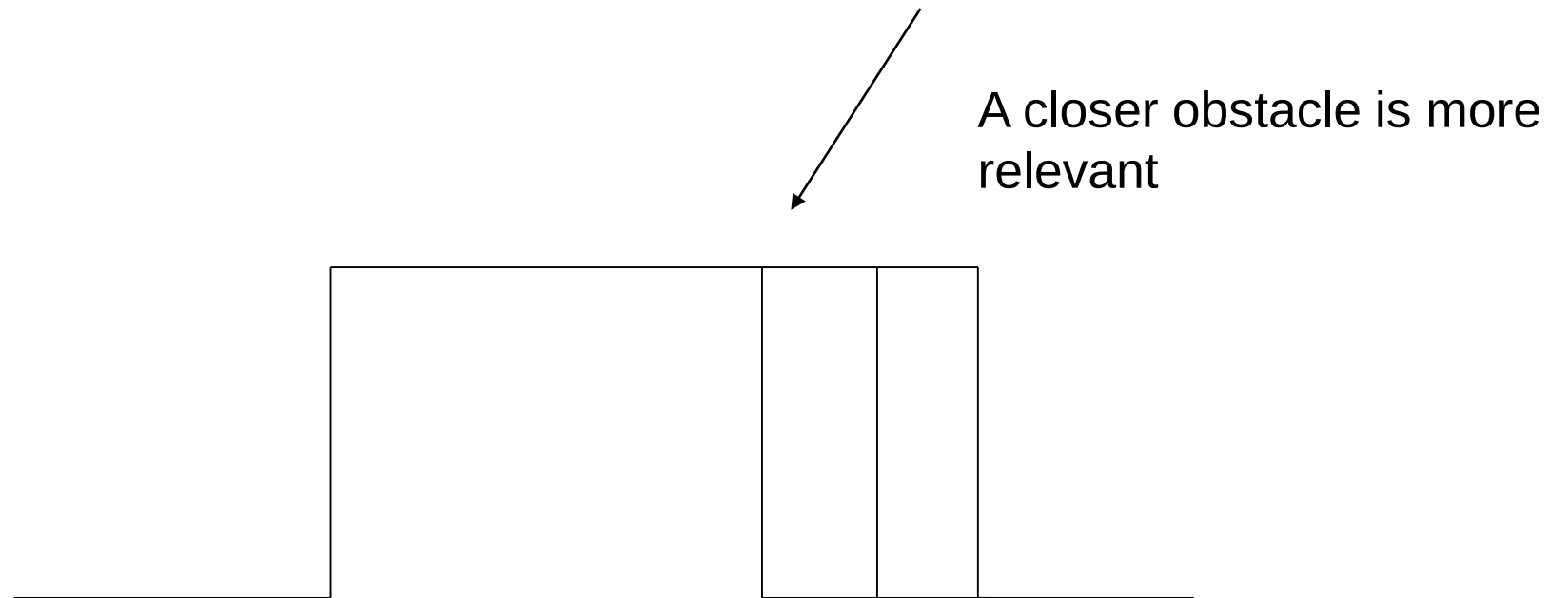
$$y_3 = \text{med}(80, 6, 2) = \text{med}(2, 6, 80) = 6,$$

$$y_4 = \text{med}(6, 2, 3) = \text{med}(2, 3, 6) = 3,$$

$$\text{i.e. } y = (3, 6, 6, 3).$$

Min/Max Filters

- Similar to median filters
- Out of a set of measured values, the min or max value is taken, depending on the relevance for the application.



Plausibility Validation

- Knowing the system and the environmental model, we sometimes can make assumptions on the plausibility of data
- E.g. a close obstacle, which is only shown sporadically, might be a small obstacle, e.g. a chair
- As a consequence sensor values get an additional plausibility value, which can be used by further processing steps.
- Often used for map building and similar applications.
- ➔ Kalman Filter

Some general remark on filters

Filters modify the original data.

This usually means that we lose information.

Carefully consider this when choosing a filter.

Application Development

We want to slow down the car if an obstacle is close to the car.

```
SIGNAL_ULTRASONICSENSOR_data_t us_front = RTE_SIGNAL_ULTRASONICSENSOR_get(&SO_USFRONT_signal);
SIGNAL_ULTRASONICSENSOR_data_t us_rear = RTE_SIGNAL_ULTRASONICSENSOR_get(&SO_USREAR_signal);

//Read the requested speed signal from the RTE
SIGNAL_SPEED_data_t targetSpeed = RTE_SIGNAL_SPEED_get(&SO_TARGETSPEEDREQUESTED_signal);

//Check for possible obstacles
if (targetSpeed.m_vy > 0)
{
    //LOG_I("CONTROL_limitSpeed_run","us_front: %d",us_front.m_distance);
    if (us_front.m_distance < CONTROL_OBSTACLE_STOP)
    {
        targetSpeed.m_vy = 0;
    }
    else if (us_front.m_distance < CONTROL_OBSTACLE_SLOW)
    {
        targetSpeed.m_vy = targetSpeed.m_vy / 2;
    }
    else
    {
        //Full speed
    }
}
else if (targetSpeed.m_vy < 0)
{
    if (us_rear.m_distance < CONTROL_OBSTACLE_STOP)
    {
        targetSpeed.m_vy = 0;
    }
    else if (us_rear.m_distance < CONTROL_OBSTACLE_SLOW)
    {
        targetSpeed.m_vy = targetSpeed.m_vy / 2;
    }
    else
    {
        //Full speed
    }
}
```

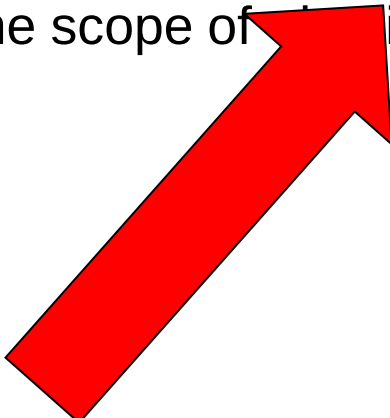
Looking back – where did the efforts go

- Getting the timer up and running was quite frustrating. Documentation was not complete, even the Cypress examples did not work or have been too trivial.
- Debugging
 - Cyclic tasks were running out of stack, creating strange errors
 - Changing the timer clock from 10 μ s tick to 1 μ s tick – only a minor beautification, but this resulted in an overrun of the 16bit timer size.
 - Bug in the RTE – for the pullport function, the parameter is not initialized. Usually no big deal, as we only return data, but in this use case I also used the parameter to read the id of the sensor
- Application code – 10min ;-)

Remember this statement

A well-defined segmentation of the project effort ensures system modularity . Each task forms a separate, distinct program module. At implementation time each module and its inputs and outputs are well defined , there is no confusion in the intended interface with other system modules. At checkout time the integrity of the module is tested independantly ; there are few scheduling problems in synchronizing the completion of several tasks before checkout can begin. Finally, the system is maintained in modular fashion; system errors and deficiencies can be traced to specific system modules, thus limiting the scope of piled error searching.

Gouthier and Pont, 1970



Stack and other resource problems pop up anywhere!

Memory

- Stack
- Heap
- Const Data
- Global Data
- ...

The top screenshot shows the 'Tasks' tab of the 'Configure ErikaOS_1' dialog. It lists tasks like 'tsk_Init', 'tsk_Background', 'TzCtrl', 'tsk_CyclicDispatcher', and 'tsk_EventDispatcher'. The 'tsk_EventDispatcher' task is selected, showing its priority (6) and schedule (Full). It also lists events and resources associated with the task.

The bottom screenshot shows the 'OS Config' tab. It includes sections for 'Basic OS Config' (CPU Type: Cortex M4, Kernel Type: ECC2, OS Status: Extended), 'Multi Stack' (checked), 'Hooks' (Startup, Shutdown, PreTask, PostTask, Error), and 'Systick' (checked, handler: systick_handler).

Option	Value
Configuration	
Device Configuration Mode	Compressed
Enable Error Correcting Code (ECC)	☐
Store Configuration Data in ECC Memory	☒
Instruction Cache Enabled	☒
Enable Fast IMO During Startup	☒
Unused Bonded IO	Allow but warn
Heap Size (bytes)	0x800
Stack Size (bytes)	0x0F00
Include CMSIS Core Peripheral Library Files	☒
Programming/Debugging	
Debug Select	GPIO
Enable Device Protection	☐
Embedded Trace (ETM)	☐
Use Optional XRES	☐
Operating Conditions	
VDDA (V)	5.0
VDDD (V)	5.0
VDDIO0 (V)	5.0
VDDIO1 (V)	5.0

```

No ELF section .cychecksum found, creating one
application checksum calculated and stored in ELF section .cychecksum
Checksum calculated and stored in ELF section .cymeta
cyelftool.exe -S D:\50_HDA\60_Lehre\40_Vorlesungen\15_EAA\20_Lecture\_work\Car\70_Car_Drive_Ultrasonic.cydsn\CortexM3\ARM_GCC_541\Debug\70_Car_Drive_Ultrasonic.elf
Flash used: 29104 of 262144 bytes (11,1 %).
SRAM used: 17353 of 65536 bytes (26,5 %). Stack: 3840 bytes. Heap: 2048 bytes.
----- Rebuild Succeeded: 07/03/2020 08:28:29 -----

```

How do we set the stacksize of the cyclic tasks?



Investigating memory

- Linker Description File ➔ Configures the memory Layout

```
PROVIDE(end = .);
PROVIDE(__bss_end__ = .);

__cy_region_init_ram = LOADADDR (.data);
__cy_region_init_size_ram = _edata - ADDR (.data);
__cy_region_zero_size_ram = _end - _edata;

/* The .stack and .heap sections don't contain any symbols.
 * They are only used for linker to calculate RAM utilization.
 */
.heap (NOLOAD) :
{
    . = _end;
    . += 0x800;
    __cy_heap_limit = .;
} >ram

.stack (__cy_stack - 0x0F00) (NOLOAD) :
{
    __cy_stack_limit = .;
    . += 0x0F00;
} >ram

/* Check if data + heap + stack exceeds RAM limit */
ASSERT(__cy_stack_limit >= __cy_heap_limit, "region RAM overflowed with stack")
```

Investigating memory

MAP-File → Shows the addresses of all globals (data and code)

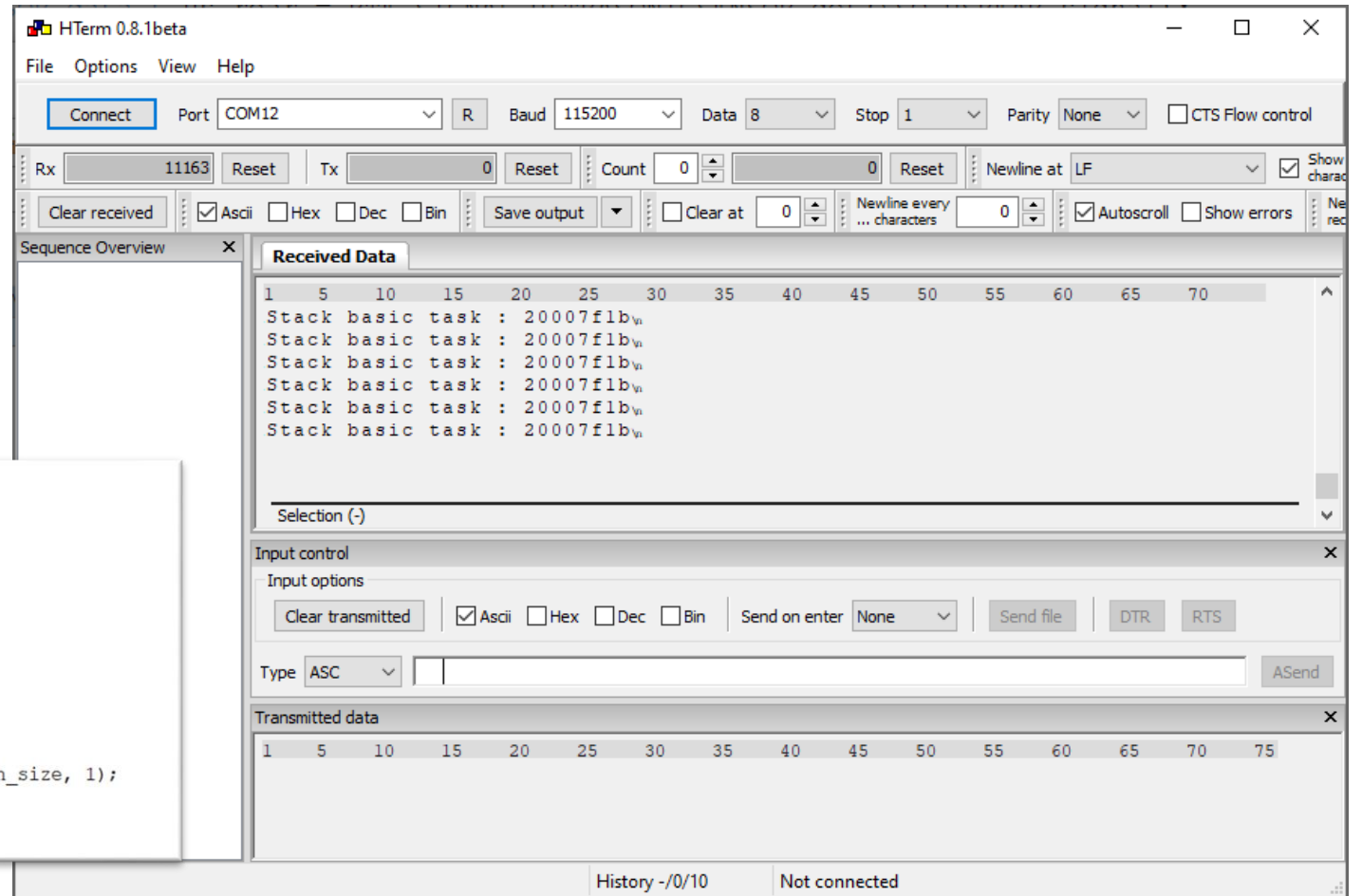
.heap	0x1ffffacd0	0x800	load address 0x000071b0
	0x1ffffacd0		. = _end
	0x1ffffb4d0		. = (. + 0x800)
fill	0x1ffffacd0	0x800	
	0x1ffffb4d0		__cy_heap_limit = .
.stack	0x20007100	0xf00	
	0x20007100		__cy_stack_limit = .
	0x20008000		. = (. + 0xf00)
fill	0x20007100	0xf00	
	0x00000001		ASSERT ((__cy_stack_lir
	0x00000000		cy_checksum_exclude_si:
	0x00000001		ASSERT ((cy_checksum_e

Basic Task Stack Allocation

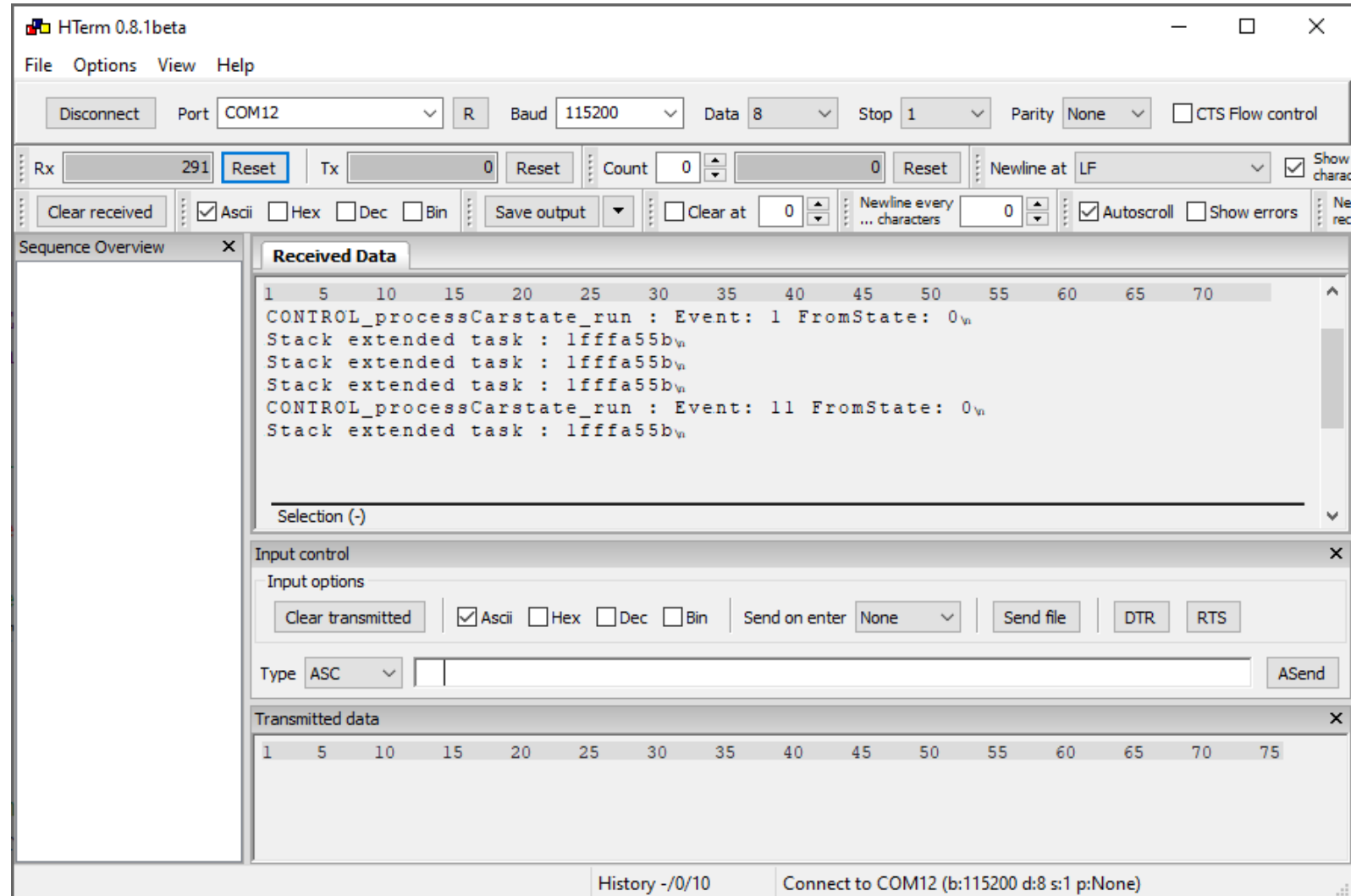
```
//Will be called every 1ms
TASK(tsk_CyclicDispatcher)
{
    //Some stack analysis
    uint8_t data = 0;
    LOG_I("Stack basic task", "%x", &data);

    //Increment the age of all signals by 1ms
    RTE_timertick();

    //Process all cyclic runnables which are due
    RTE_ProcessCyclicTable(RTE_cyclicActivationTable, RTE_cyclicActivation_size, 1);
}
```

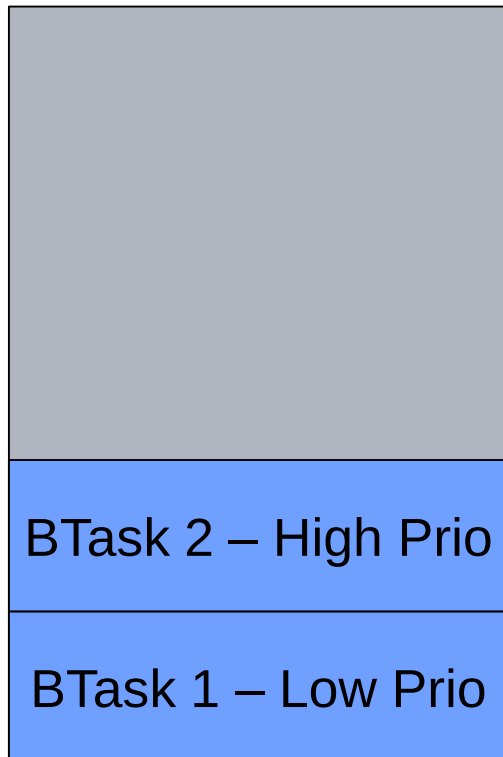


Extended Task Stack Allocation

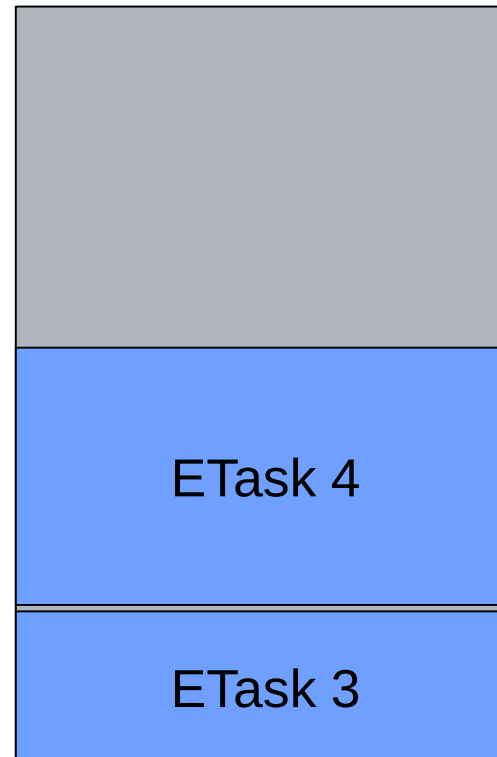


The (almost) complete stack model for Erika

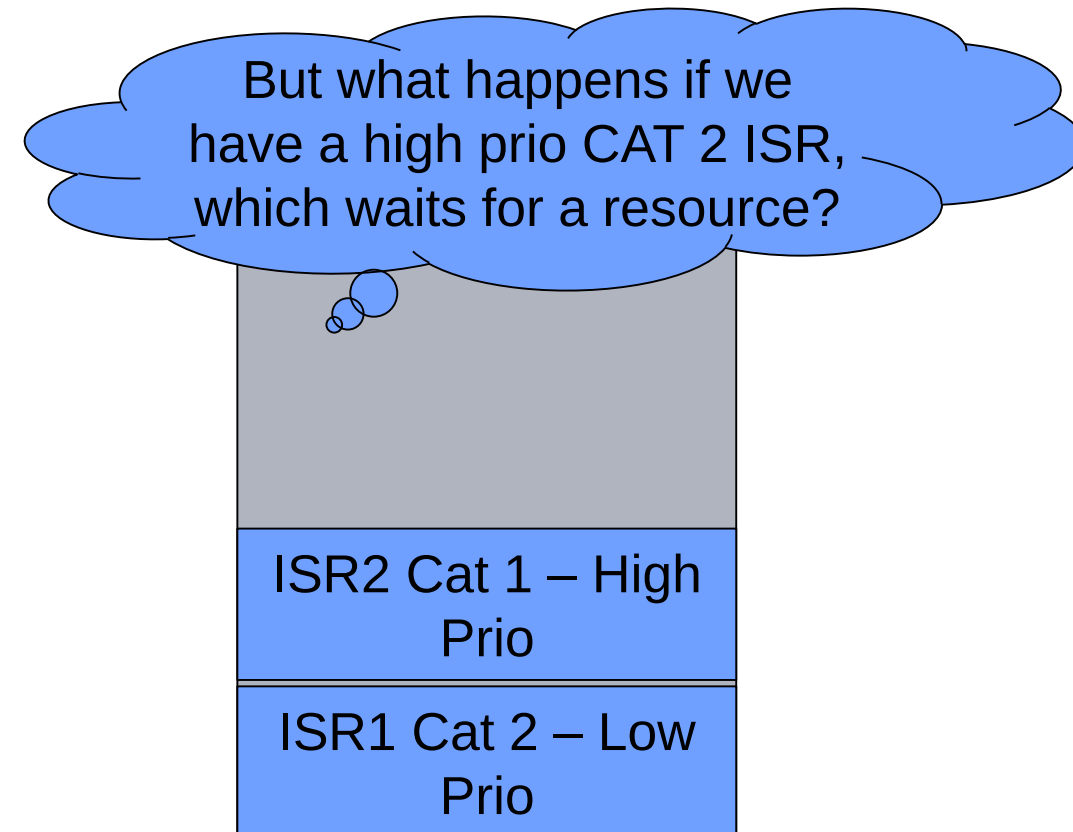
Basic Tasks



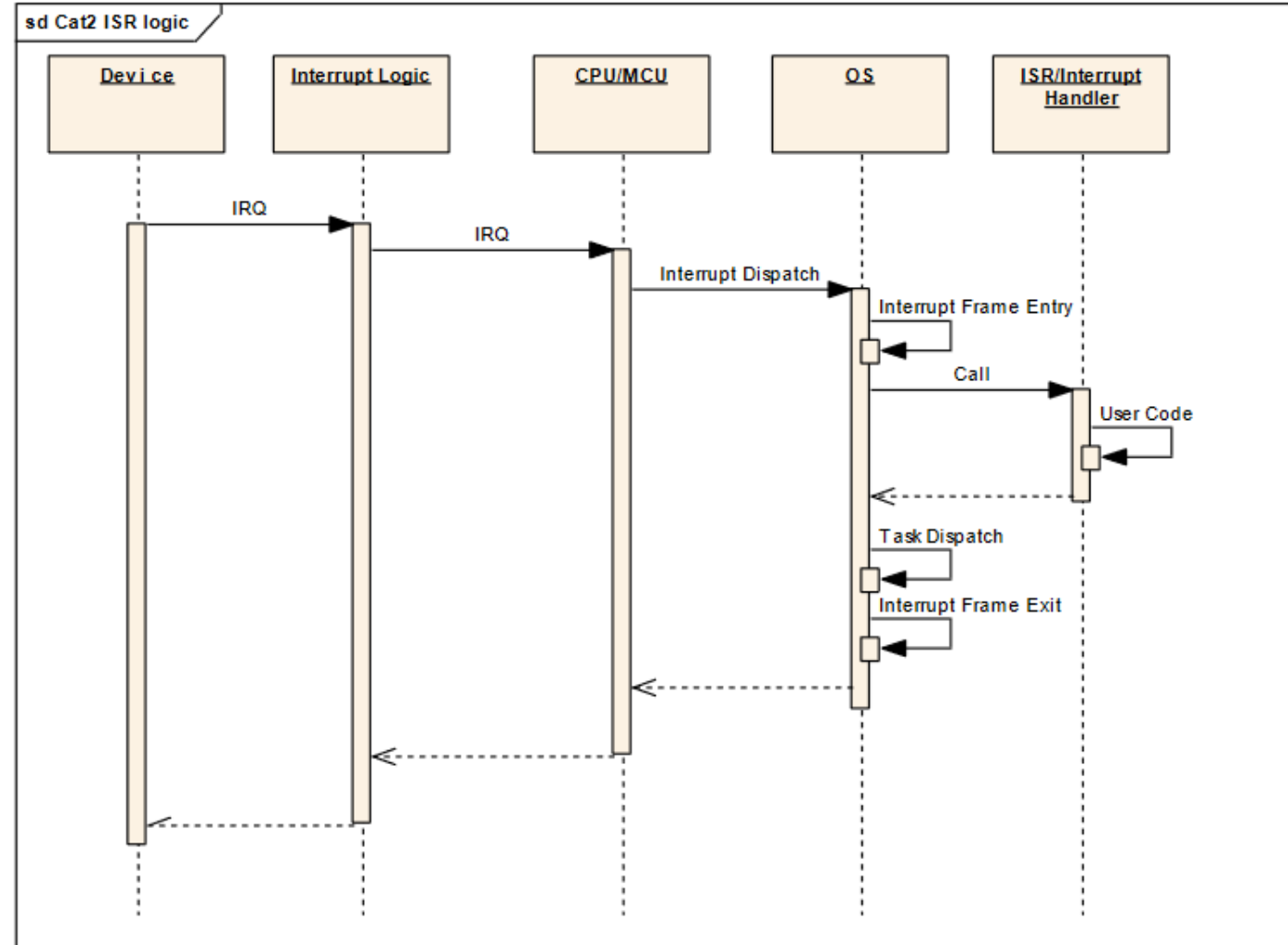
Extended Tasks



Interrupts



ISR Category 2



Wrap-Up

- Comparison of a synchronous and asynchronous driver implementation
 - Synchronous implementation was straight forward but blocks too long – no option
 - Asynchronous has a much better timing behavior but is way more complex (and fragile) in its implementation
- Resource bugs (like running out of stack) are hard to debug
 - We have to understand the memory configuration
 - And deal with parts of the build process, we ignored so far
 - Linker Description File
 - MAP File